Lecture 2
Intro. To Software Engineering and Object-Oriented Programming (1/2)

Bong-Soo Sohn
Chung-Ang University

What is a Good Program? (Factors of Good Software)

Software Crisis

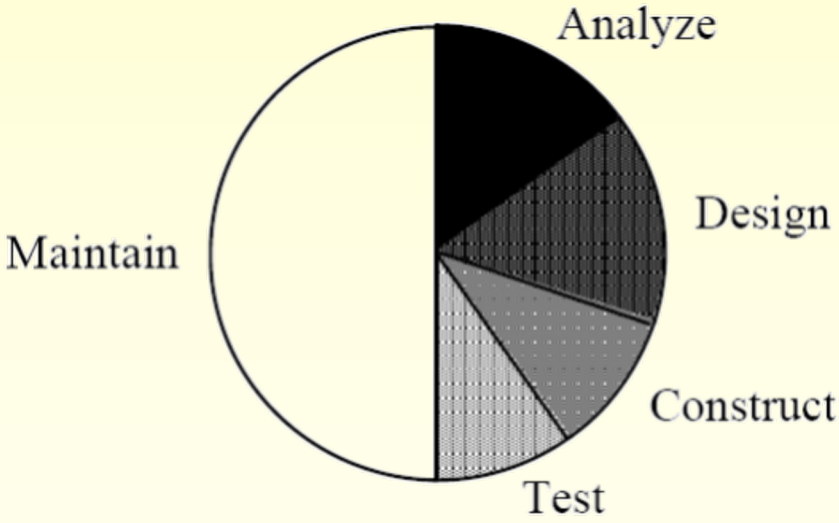
- rapid increases in computer power and the complexity of the problems (SW).
- Software technology can not follow the complexity.

- the difficulty of writing correct, understandable, and verifiable computer programs

- SW Symptoms
  - Unmanageable
  - difficult to maintain
  - Over-budget.
  - Over-time (late)
  - Poor quality
  - Software often did not meet requirements.
- Cause : complexity

Software Life Cycle

1. Requirement Analysis
   - Problem specification
2. Design
   - Program structure
   - Module specification
3. Implementation
4. Test
   - unit test
   - integration test
5. Maintenance



Fred Brooks states that over 90% of the costs of a typical system arise in the maintenance phase, and that any successful piece of software will inevitably be maintained.

Software Engineering

Definition

- [IEEE Definition] The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software
  , that is, application of engineering to software

- [Firtz Bauer] The establishment and use of sound engineering principles in order to economically obtain software that is reliable and works efficiently

Goals

- produce a correct, efficient, and reliable software
- minimize the cost of software development and maintenance

Goals of SE

- Maintainability
  - the ability to easily make changes, enhancements, or improvements.
- Dependability
  - the ability to rely on the software to function properly when needed.
- Efficiency
  - the ability for software to use computing resources effectively (mainly space and time).
- Usability
  - the ability for the end user to easily and effectively put the software to proper use.

Software Properties

- **Modularity** – divide and conquer.
- **Encapsulation** – hide the implementation.
- **Localization** – collect similar things together.
- **Abstraction** – simplifying complex reality.
- **Uniformity** – make everything look similar.
- **Completeness** – do everything required.
- **Confirmability** – be able to prove that the software works properly.

Programming Methodology

- Unstructured Programming
- Procedural Programming
- Modular(Structured) Programming
- Object Oriented Programming

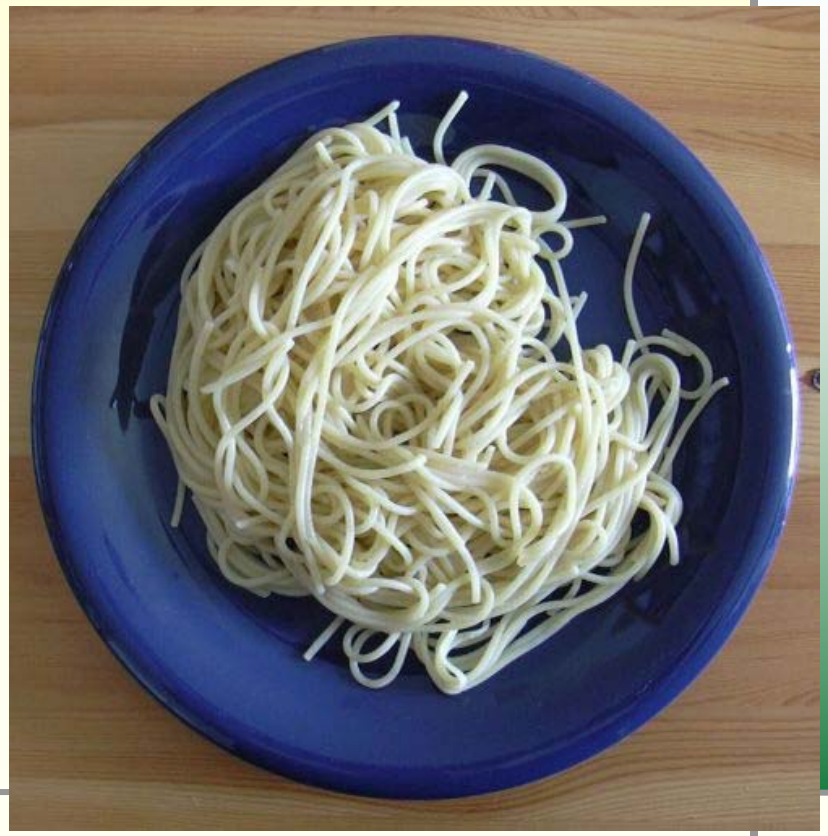
Unstructured Programming

- Consists of only one main program.
- a sequence of commands or statements
- Only global data
- Use goto statement

- Disadvantage
  - If there is the same statement sequence?

Spaghetti Code

- Code with complex and tangled structure
  - With goto's jumping all over the place
  - Almost impossible to understand
  - Hard to follow the flow
  - Difficult to modify



Procedural Programming

- The abuses of spaghetti code led to structured programming languages to support **procedural programming**

- Program
  - A sequence of procedure calls
- Procedure
  - the same seqeunce of statements

- More structured : reduce repetition
- More error-free

- Structured Programming
  - Top-down approach
  - Divide and conquer
  - Functional decomposition

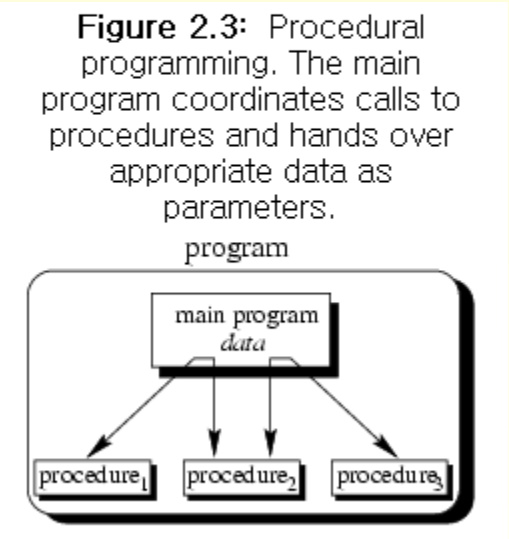


Figure 2.3: Procedural programming. The main program coordinates calls to procedures and hands over appropriate data as parameters.

Modular Programming

- SW is composed of separate parts (module)
- procedures of a common functionality are grouped together into separate *modules*
- module
  - Program uses modules through interface.
  - Interface : visible, implementation : hidden
  - Improve maintainability

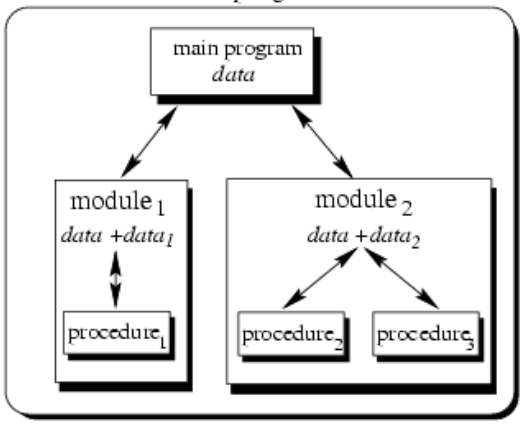
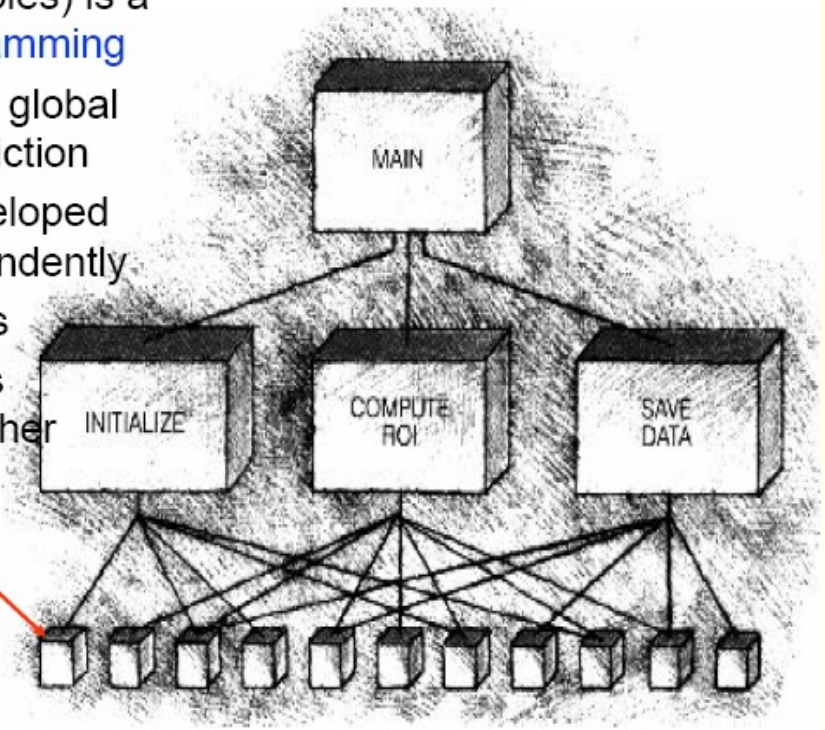


Figure 2.4: Modular programming. The main program coordinates calls to procedures in separate modules and hands over appropriate data as parameters.

Don't use Global Variables

- Using global variables : unsafe!

- Sharing data (global variables) is a violation of modular programming
- All modules can access all global variables without any restriction
  - No module can be developed and understood independently
- Global Data are dangerous
  - This makes all modules dependent on one another

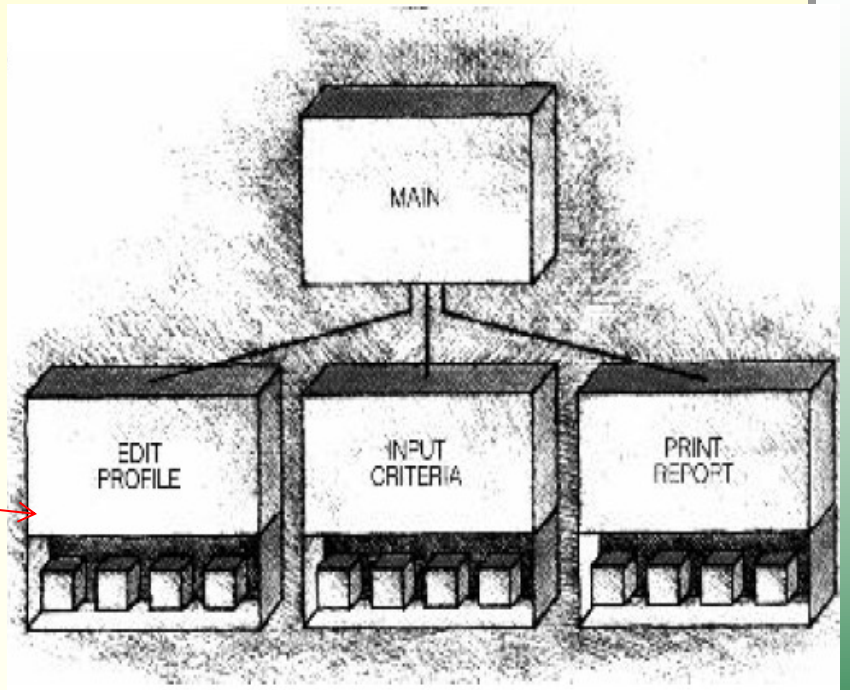


Copyright: OOT A Managers' perspective, Dr. Taylor

Modularized Data

- Localize data inside the modules
- This makes modules more independent of one another
- Safe!

Local data



Copyright: OOT A Managers' perspective, Dr. Taylor

Problems of Procedural/Modular Programming

- Decoupled (seperation of) data and operations
- Doesn't work well in large systems
  - Difficult to debug, extend, and maintain
  - Code reuse limited
- Explicit creation and destruction
- Many others compared to OOP

OOP (Object Oriented Programming)

- Program consists of a set of interacting objects.
- Each object keeps its own state.
- Combine data and operation into an object

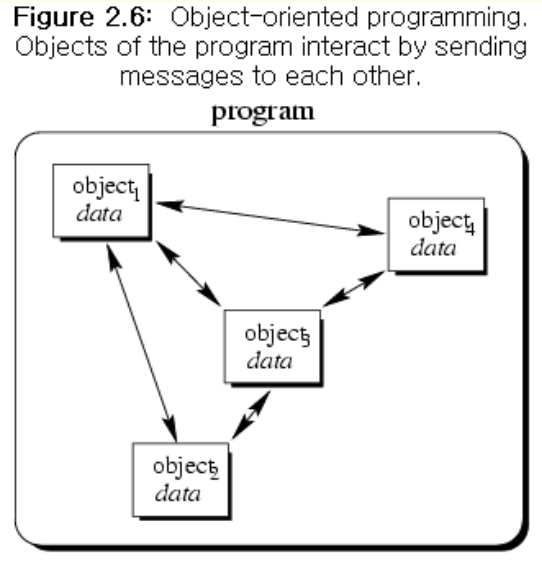


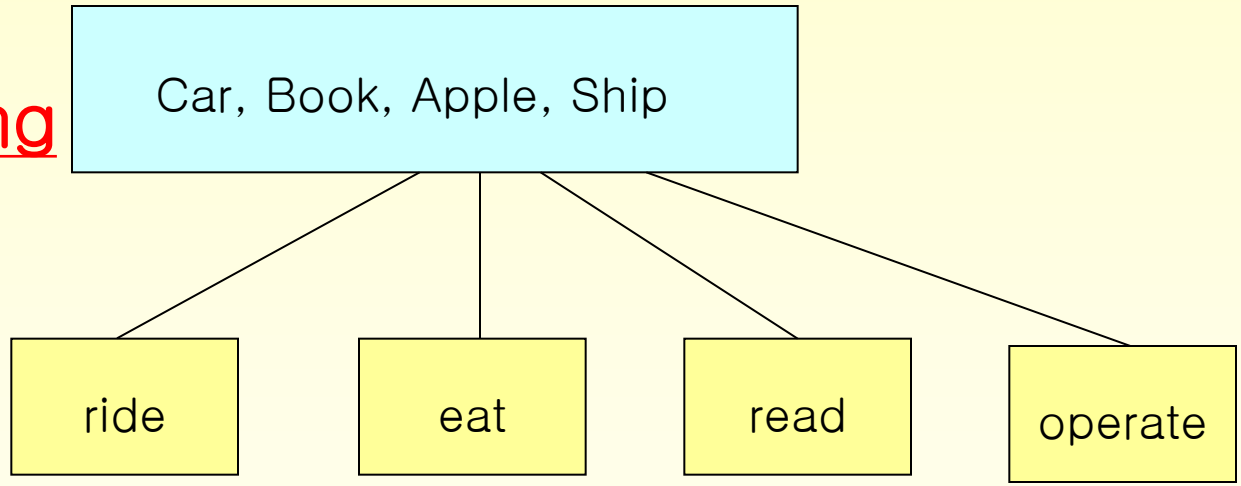
Figure 2.6: Object-oriented programming. Objects of the program interact by sending messages to each other.

What is an Object?

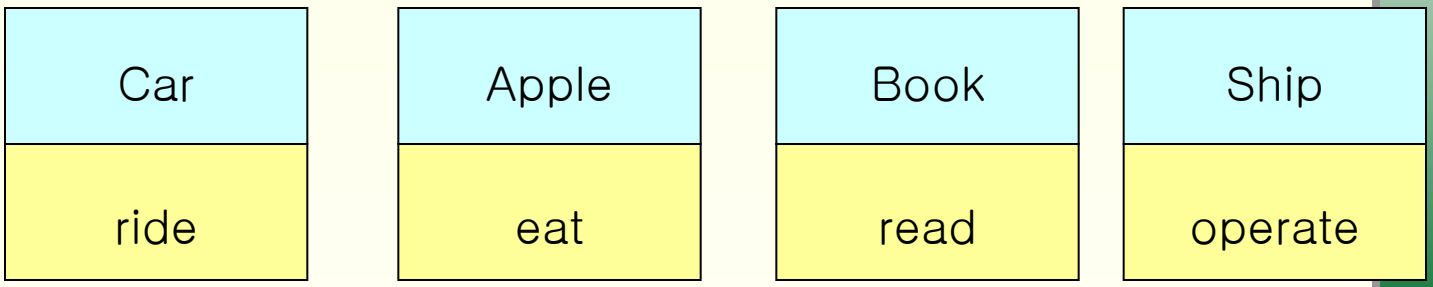
- Object is an instance of a class
- Objects have
  - identity
  - attribute
  - behavior (method)
  - relationships with other objects

Structured Programming vs OOP

Structured Programming



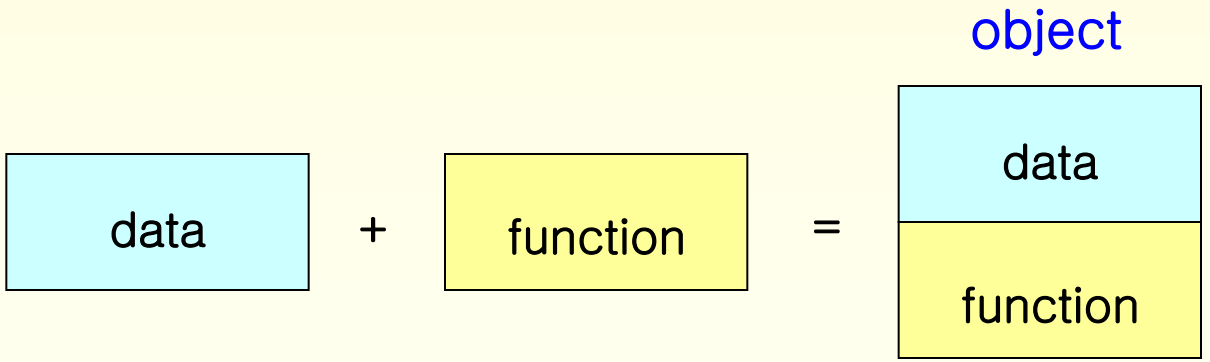
OOP



Structured Programming vs OOP

Procedural Programming

data + function = program



OOP

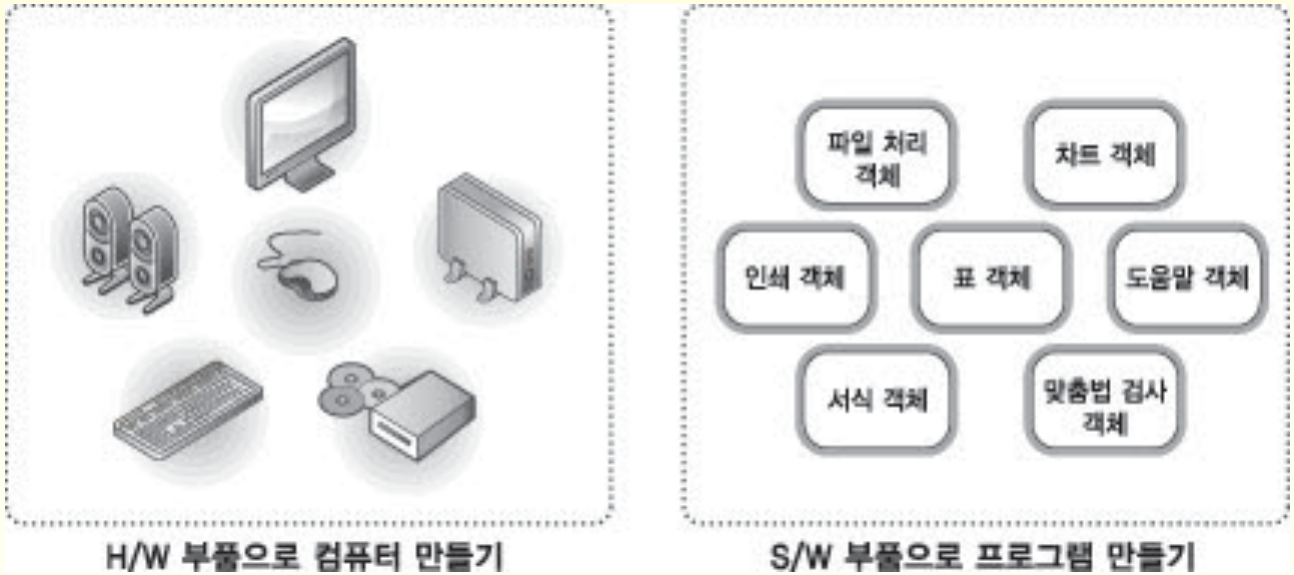
object + object = program

Views of OOP

- OOP does not consider the problem domain as just a flow of processings.
- OOP defines objects related to the problem domain and regards these objects communicate information each other (through message passing).
- On the other hand, structured programming treats data and operations independently even though they are related.

Software Components(부품화)

- In OO programs, an object works as a component(부품) of software
- Interface of an object should be well defined.



H/W 부품으로 컴퓨터 만들기

S/W 부품으로 프로그램 만들기

Why Object Oriented?

- Good for developing large software
- Maintainability
- Reuse software, not rewrite
- Increase
  - Programmer productivity
  - Quality of software
  - Understandability of software
  - Scalability (extensibility,reusability) of software
  - Lifetime (maintainability) of software

OOP

- What is problem?
  - Traditional structured programming is not sufficient for large software maintenance and reuse.
- Solution? OOP
  - Improve reusability
  - SW reliability is increased using reusable modules which are already well-tested and error-free .

OOP concepts

- Data Abstraction
  - Abstract Data Type
- Object
  - Information hiding/encapsulation
- Class
- Inheritance
- Polymorphism
- Dynamic binding

ADT(Abstract Data Type)

- A set of data variables and associated operations that are precisely specified independent of any particular implementation.
- ADT = data + operations (no implementation)
- In OOP,
  - We don't define data for defining procedures,
  - But we do define procedures necessary for defining data
  - ADT is a type that combines data and the procedures

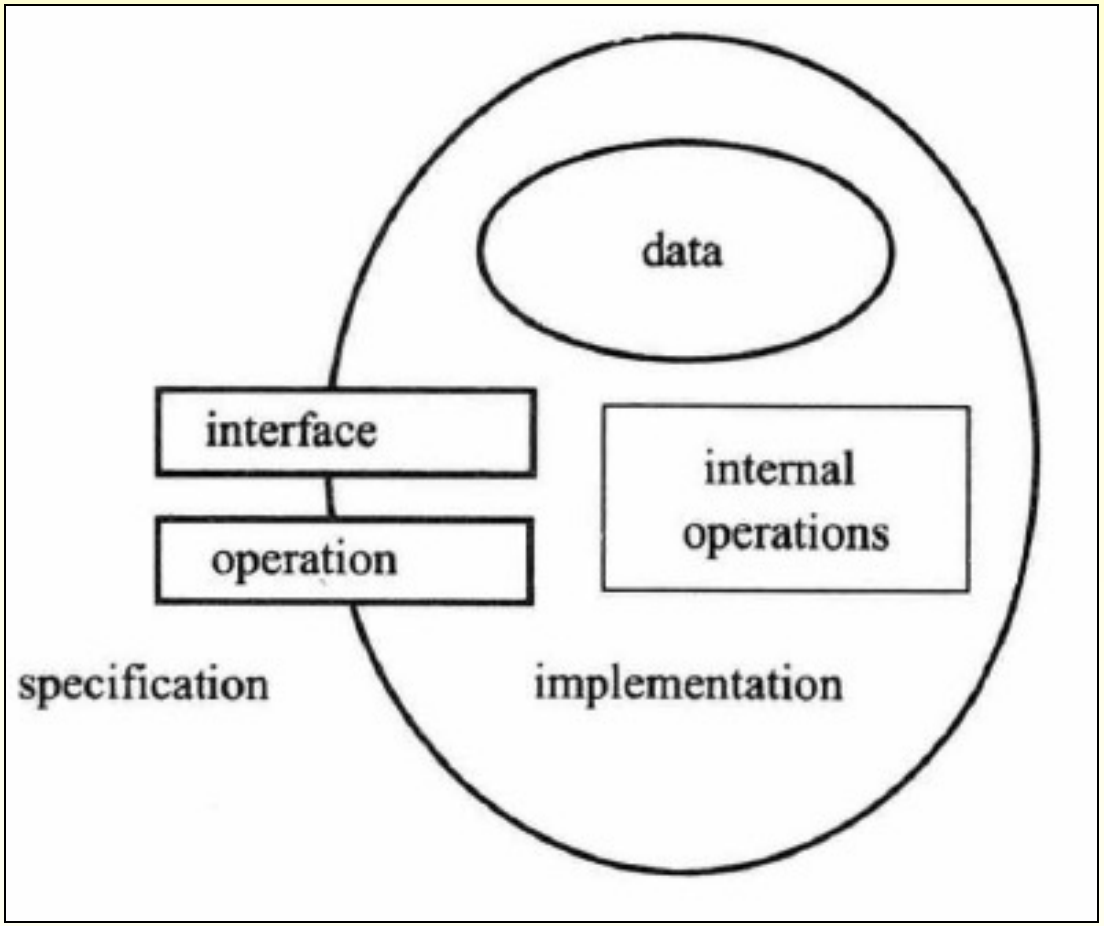
class

- In C++, “class” is used for defining ADT
- Defines attributes and behaviors of an object
  - Attributes : member variables
  - Behaviors : member functions

Object

- Object is an instance of class
- Consists of
  - Internal data
  - Internal operations
  - Interface operations
- Only class methods can access the internal data of an object.
- Message passing
  - Request some work to other object through calling a method.

Object Overview



Information Hiding/Encapsulation

- Encapsulation conceals the functional details of a class from objects that send messages to it.
- Internal data of an object can be modified only through an interface

Class vs Object

- Each object generated by the same class is different objects
  - When the objects are created, different memory space and address are assigned to each object with unique identifier.
- Objects created by the same class share the class's methods.
- An object can be dynamically constructed or destructed in memory during run-time.

Inheritance

- A child class (Sub class) inherits attributes and behaviors of parent classes (super class).
- and can define their own attributes and behaviors.

- The properties (variables,methods) of a child class should be mostly similar to the properties of parent classes.

- Goal : Object (class) Reuse
- In large SW development, code reuse reduces costs.
- We can reuse well-defined classes.
- Related programs can define new classes using inheritance.

Polymorphism

- Allows the same method (message) to different meaning.
- the ability of objects belonging to different data types to respond to method calls of the same name, each one according to an appropriate type-specific behavior.
- In C++
  - Virtual function
  - Operator overloading
  - Function overloading
  - Template

Benefits of Object Model

- Builds a system that evolves over time
  - Entendible
  - Stable
- Thinking in terms of objects and classes is much easier for humans
- Separating the client and implementor prevents accidental damage through data encapsulation
- Reusability

보충 자료 (강의 페이지)

- Handout 1: Factors of Good Software

Handout 1: Factors of Good Software

OOP Handout 1 : Factors of Software Quality

- **What is a good program? (Factors of Good Software)**
  - correct (correctness)
    - satisfy user requirement/specification without error
  - efficient (efficiency)
    - optimizes resource usage to deliver fast response times
    - space / time , HW utilization
  - easy to maintain (maintainability)
    - easy to read (readability)
    - easy to modify (modifiability)
    - easy to extend(extensibility) , scalability
      - scalability: handles growing volumes of users, data, or concurrent tasks smoothly by adding infrastructure resources.
    - easy to reuse (reusability)
    - easy to debug , easy to test (testability)
    - modular (modularity)
    - easy to port (portability) , platform-independent (HW, OS, …)
  - stable (stability), reliable (reliability) , robust (robustness)
  - easy to use (usability) , easy to run
  - secure (security) , integrity (accurate & unaltered data, protection)
    - Protects data against unauthorized access, corruption, or malicious exploits.
  - Interoperability
    - Easily exchanges data and integrates with other software systems via standardized protocols and interfaces.
  - concise (conciseness)
    - writing code, designing architectures, and crafting interfaces that achieve maximum result with the minimum necessary complexity or syntax without sacrificing readability or correctness. (ex) minimized redundancy. simplified branch decisions
  - complete (completeness)
    - implements all required functionality
  - consistent (consistency)
    - Uniformity of design, behavior, coding standards, and user interfaces across an entire system.
    - ex) uniform naming conventions (e.g., camelCase vs snake_case), formatting
  - well-documented (documentation)
  - well structured